JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class DeflaterInputStream

java.lang.Object
  java.io.InputStream
    java.io.FilterInputStream
      java.util.zip.DeflaterInputStream

All Implemented Interfaces:

`Closeable`, `AutoCloseable`

```
public class DeflaterInputStream
extends FilterInputStream
```

Implements an input stream filter for compressing data in the "deflate" compression format.

Compressor Usage

A `DeflaterInputStream` created without specifying a compressor will create a compressor at construction time, and close the compressor when the input stream is closed.

If a compressor is specified when creating a `DeflaterInputStream`, it is the responsibility of the caller to close the compressor after closing the input stream.

API Note:

The `close()` method should be called to release resources used by this stream, either directly, or with the `try`-with-resources statement.

Since:

1.6

See Also:

`DeflaterOutputStream`, `InflaterOutputStream`, `InflaterInputStream`

Field Summary

Fields

Modifier and Type	Field	Description
`protected final byte[]`	**`buf`**	Input buffer for reading compressed data.
`protected final Deflater`	**`def`**	Compressor for this stream.

Fields declared in class FilterInputStream
in

Constructor Summary

Constructors

Constructor	Description
DeflaterInputStream(InputStream in)	Creates a new input stream and compressor with the default compression level and a default buffer size.
DeflaterInputStream(InputStream in, Deflater defl)	Creates a new input stream with the specified compressor and a default buffer size.
DeflaterInputStream(InputStream in, Deflater defl, int bufLen)	Creates a new input stream with the specified compressor and buffer size.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
int	**available**()	Returns 0 after EOF has been reached, otherwise always return 1.
void	**close**()	Closes this input stream and its underlying input stream, discarding any pending uncompressed data.
void	**mark**(int limit)	*This operation is not supported*.
boolean	**markSupported**()	Always returns `false` because this input stream does not support

		the `mark()` and `reset()` methods.
`int`	**`read`**`()`	Reads a single byte of compressed data from the input stream.
`int`	**`read`**`(byte[] b, int off, int len)`	Reads compressed data into a byte array.
`void`	**`reset`**`()`	*This operation is not supported*.
`long`	**`skip`**`(long n)`	Skips over and discards data from the input stream.

Methods declared in class FilterInputStream

`read`

Methods declared in class InputStream

`nullInputStream`, `readAllBytes`, `readNBytes`, `readNBytes`, `skipNBytes`, `transferTo`

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Field Details

def

```
protected final Deflater def
```

Compressor for this stream.

buf

```
protected final byte[] buf
```

Input buffer for reading compressed data.

Constructor Details

DeflaterInputStream

```
public DeflaterInputStream(InputStream in)
```

Creates a new input stream and compressor with the default compression level and a default buffer size.

The compressor will be closed when this input stream is closed.

Parameters:

`in` - input stream to read the uncompressed data to

Throws:

`NullPointerException` - if `in` is null

DeflaterInputStream

```
public DeflaterInputStream(InputStream in,
                           Deflater defl)
```

Creates a new input stream with the specified compressor and a default buffer size.

Closing this input stream will not close the given compressor.

Parameters:

`in` - input stream to read the uncompressed data to

`defl` - compressor ("deflater") for this stream

Throws:

`NullPointerException` - if `in` or `defl` is null

DeflaterInputStream

```
public DeflaterInputStream(InputStream in,
                           Deflater defl,
                           int bufLen)
```

Creates a new input stream with the specified compressor and buffer size.

Closing this input stream will not close the given compressor.

Parameters:

`in` - input stream to read the uncompressed data to

`defl` - compressor ("deflater") for this stream

`bufLen` - compression buffer size

Throws:

`IllegalArgumentException` - if `bufLen <= 0`

`NullPointerException` - if `in` or `defl` is null

Method Details

close

```
public void close()
           throws IOException
```

Closes this input stream and its underlying input stream, discarding any pending uncompressed data.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Overrides:

`close` in class `FilterInputStream`

Throws:

`IOException` - if an I/O error occurs

See Also:

`FilterInputStream.in`

read

```
public int read()
         throws IOException
```

Reads a single byte of compressed data from the input stream. This method will block until some input can be read and compressed.

Overrides:

`read` in class `FilterInputStream`

Returns:

a single byte of compressed data, or -1 if the end of the uncompressed input stream is reached

Throws:

`IOException` - if an I/O error occurs or if this stream is already closed

See Also:

`FilterInputStream.in`

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Reads compressed data into a byte array. This method will block until some input can be read and compressed.

Overrides:

`read` in class `FilterInputStream`

Parameters:

`b` - buffer into which the data is read

`off` - starting offset of the data within `b`

`len` - maximum number of compressed bytes to read into `b`

Returns:

the actual number of bytes read, or -1 if the end of the uncompressed input stream is reached

Throws:

`NullPointerException` - if `b` is null

`IndexOutOfBoundsException` - if `len > b.length - off`

`IOException` - if an I/O error occurs or if this input stream is already closed

See Also:

`FilterInputStream.in`

skip

```
public long skip(long n)
          throws IOException
```

Skips over and discards data from the input stream. This method may block until the specified number of bytes are skipped or end of stream is reached.

Overrides:

`skip` in class `FilterInputStream`

Implementation Note:

This method skips at most `Integer.MAX_VALUE` bytes.

Parameters:

n - number of bytes to be skipped. If n is zero then no bytes are skipped.

Returns:

the actual number of bytes skipped, which might be zero

Throws:

IOException - if an I/O error occurs or if this stream is already closed

IllegalArgumentException - if n < 0

See Also:

InputStream.skipNBytes(long)

available

```
public int available()
             throws IOException
```

Returns 0 after EOF has been reached, otherwise always return 1.

Programs should not count on this method to return the actual number of bytes that could be read without blocking

Overrides:

available in class FilterInputStream

Returns:

zero after the end of the underlying input stream has been reached, otherwise always returns 1

Throws:

IOException - if an I/O error occurs or if this stream is already closed

markSupported

```
public boolean markSupported()
```

Always returns false because this input stream does not support the mark() and reset() methods.

Overrides:

markSupported in class FilterInputStream

Returns:

false, always

See Also:

FilterInputStream.in, InputStream.mark(int), InputStream.reset()

mark

```
public void mark(int limit)
```

This operation is not supported.

Overrides:

`mark` in class `FilterInputStream`

Parameters:

`limit` - maximum bytes that can be read before invalidating the position marker

See Also:

`FilterInputStream.in`, `FilterInputStream.reset()`

reset

```
public void reset()
           throws IOException
```

This operation is not supported.

Overrides:

`reset` in class `FilterInputStream`

Throws:

`IOException` - always thrown

See Also:

`FilterInputStream.in`, `FilterInputStream.mark(int)`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.